TP3 :Login avec Middleware – Application FullStack

Dr. Nesrine MEDDEB

Objectif

- Implémenter un système d'authentification basé sur :
- Login utilisateur
- Génération d'un token JWT
- Middleware de vérification
- Accès à une route protégée

Pourquoi on utilise JWT?

- On utilise **JWT (JSON Web Token)** principalement pour gérer **l'authentification** de manière simple, sécurisée et stateless.
- Quand l'utilisateur se connecte :
- Il envoie email + password
- Le serveur vérifie
- Le serveur génère un **token JWT**
- Le client garde ce token
- À chaque requête suivante → il renvoie le token
- 👉 Le serveur sait que l'utilisateur est authentifié grâce au token.

Pourquoi on utilise JWT?

- C'est le point clé.
- Avec JWT :
- Le serveur **ne stocke pas la session**
- Toutes les infos sont dans le token
- Contrairement aux sessions classiques :

Session classique	**JWT**
Stockée côté serveur	Rien stocké côté serveur
Besoin de mémoire serveur	Scalabilité facile
Moins pratique en microservices	Parfait pour API REST

Pourquoi on utilise JWT?

- Un JWT peut contenir :
- id utilisateur
- rôle (admin / user)
- date d'expiration
- Exemple payload :
- {
- "id": "6587abc...",
- "role": "admin",
- "exp": 1712345678
- }

Donc à chaque requête :
Pas besoin d'aller chercher l'utilisateur en base pour savoir son rôle.
Le middleware peut lire le token directement.

Travail demandé

- Partie 1
- Créer un API Login (POST /users/login) qui permet de:
  1) Vérifier si l'utilisateur existe (email)
  2) Comparer le mot de passe avec bcrypt.compare
  3) Si correct :

     Générer un JWT contenant : id, username
  4) Retourner:

     token

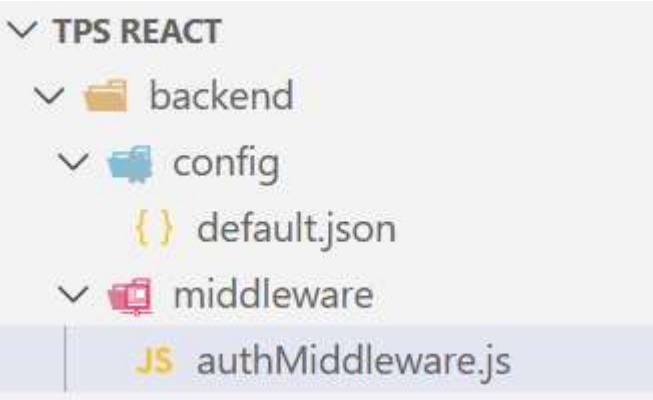
     username
- Si email ou password incorrect → retourner erreur.

```
// Login
router.post('/login', async (req, res) => {
  const { email, password } = req.body;

  try {
    const user = await User.findOne({ email });
    if (!user) return res.status(400).json
    ({ message: 'Utilisateur non trouvé' });

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) return res.status(400).json({ message: 'Mot de passe incorrect' });
    const JWT_SECRET=config.get("jwtSecret");
    const tokenExpire=config.get("tokenExpire");
    const token = jwt.sign({ id: user._id, name: user.username }, JWT_SECRET,
      { expiresIn: tokenExpire });

    res.json({ token, username: user.username });
  } catch (err) {
    res.status(500).json({ message: 'Erreur serveur' });
  }
});
```

Partie 2: Middleware verifyToken

- Créer un middleware :
- Il doit :
- Récupérer le token depuis :
  - Authorization header : Bearer token
- Vérifier le token avec jwt.verify()
- Ajouter les informations utilisateur dans req.user
- Si token invalide → 401 Unauthorized

Correction Partie 2: Middleware verifyToken



Correction Partie 2: Middleware verifyToken

```
1   const jwt = require('jsonwebtoken');
2   const config = require("config");
3
4   const authMiddleware = (req, res, next) => {
5       const JWT_SECRET=config.get("jwtSecret");
6       const token = req.headers.authorization?.split(' ')[1]; // Bearer <token>
7     if (!token) return res.status(401).json({ message: 'Accès refusé' });
8
9     try {
10      const decoded = jwt.verify(token, JWT_SECRET);
11      req.user = decoded; // contient id et name
12      next();
13    } catch (err) {
14      res.status(401).json({ message: 'Token invalide' });
15    }
16  };
17
18  module.exports = authMiddleware;
```

Partie 3: Route protégée

- Créer une route (dans users.js) :
- GET /home

Correction Partie 3: Route protégée

```
// Route protégée /home
router.get('/home', authMiddleware, (req, res) => {
  res.json({ message: `Bienvenue ${req.user.name} sur la page Home!` });
});
```

Partie : Frontend (React)

- **Page Login**
- Créer un formulaire :
  - email
  - password
- Envoyer les données avec axios vers :

POST http://localhost:3000/users/login

- Si succès :
  - Stocker le token et username dans localstorage
  - Rediriger vers dashboard

```
import React, { useState } from "react";
import axios from "axios";

function Login() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");

  const handleSubmit = async (e) => {
    e.preventDefault();

      try {
     const res = await axios.post("http://localhost:3001/users/login",
      { email, password }
     );
```

```
    localStorage.setItem("token", res.data.token);
    localStorage.setItem("name", res.data.username);
  } catch (err) {
    setError(err.response?.data.message || "Erreur serveur");
  }
};

    return (
  <div >
    <h2>Login</h2>
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        placeholder="Email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}  required/>
```

```
            <br /><br />

            <input
        type="password"
        placeholder="Mot de passe"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        required
      />

            <br /><br />

            <button type="submit">Se connecter</button>
    </form>

          {error && <p style={{ color: "red" }}>{error}</p>}
  </div>
 );
}


    export default Login;
```

localhost:3000/login

Login

test@gmail.com

•••

Se connecter

Welcome | Elements | Console | Sources | Network | Performance | Memory | Application

Application
- Manifest
- Service workers
- Storage

Storage
- Local storage
  - http://localhost:3000
- Session storage
- Extension storage

Filter

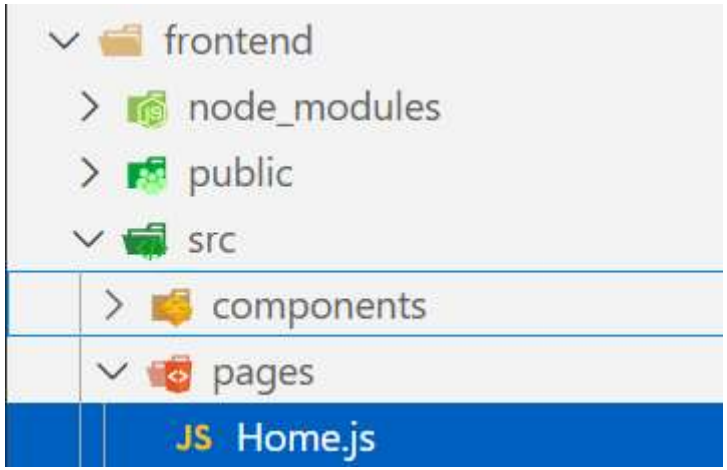
http://localhost:3000

Key	Value
loggedIn	true
name	test
token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY5ODhmNDNkOGI5MThhOThiYzUzYjg...
userRole	Admin

Partie : Frontend (React)

- **Page Home**
- Au chargement :
- Vérifier si token existe
- Envoyer requête vers /home avec header :
  - Authorization: Bearer token

Si erreur → redirection login.



```
import React, { useEffect, useState } from "react";
import axios from "axios";
import Navbar from "../components/Navbar";
import { useNavigate } from "react-router-dom";

  function Home() {

    const [message, setMessage] = useState("");
 const navigate = useNavigate();
 const token = localStorage.getItem("token");

    useEffect(() => {
  if (!token) {
    navigate("/login");
    return;
  }
```

```
        axios.get(
      "http://localhost:3001/users/home",
      {
        headers: {
          Authorization: `Bearer ${token}`
        } } )
    .then(res => setMessage(res.data.message))
    .catch(() => navigate("/login"));

        }, [token, navigate]);

        return (
    <div>

            <h2 style={{ textAlign: "center", marginTop: "50px" }}>
        {message}
      </h2>
    </div>
  );
}

      export default Home;
```

Modifier App.js pour définir les routes

```
import React from "react";
import { BrowserRouter as Router, Routes, Route, Navigate } from "react-router-dom";
import Login from "./components/Login";
import Home from "./pages/Home";

 function App() {

   const PrivateRoute = ({ children }) => {
  const token = localStorage.getItem("token");
  return token ? children : <Navigate to="/login" />;
 };

   return (
  <Router>
    <Routes>
      <Route path="/login" element={<Login />} />
```

Modifier App.js pour définir les routes

```
            <Route
          path="/home"
          element={
            <PrivateRoute>
              <Home />
            </PrivateRoute>
          }
        />

            <Route path="*" element={<Navigate to="/login" />} />

      </Routes>
    </Router>
  );
}

     export default App;
```

Bienvenue test sur la page Home!